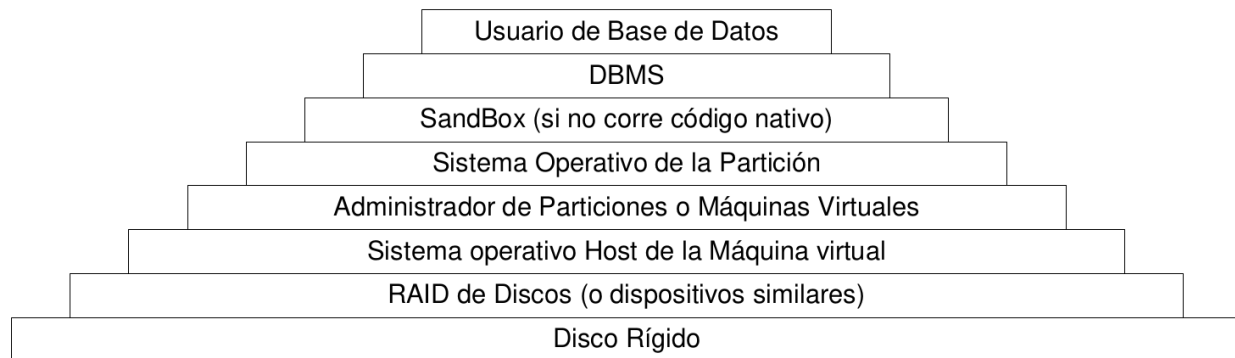


Almacenamiento físico

El objetivo de este texto no consiste en investigar a fondo las técnicas de almacenamiento ni de indexación, sino más bien de proveer una base para el entendimiento de las problemáticas de almacenamiento y las técnicas de sizing. De la misma manera, no se pretende demostrar la eficiencia o los costos de acceso de las diversas técnicas de indexación, sino más bien de presentarlas para dar al lector un contexto válido en el cuál trabajar con los mismos.

Los DBMS necesitan almacenar grandes cantidades de datos, los mismos, generalmente son almacenados en discos rígidos o dispositivos similares. Para lograr la persistencia de los archivos, generalmente los DBMS utilizan archivos del sistema operativo sobre el cual está almacenada la base de datos. No nos vamos a adentrar en cómo los datos son almacenados en un disco rígido, ya que se supone que el lector tendrá conocimientos de arquitectura de computadoras y conocimientos específicos sobre dispositivos de almacenamiento. Por otro lado, tampoco conviene describir una arquitectura en particular, ya que hoy en día las capas de abstracción antes de llegar a un dispositivo físico son muchas, por lo cual, si un motor de base de datos garantiza que almacena registros contiguos en diferentes discos rígidos y a su vez en sectores de discos contiguos, es porque tiene dominio de la arquitectura sobre la cual está montada y probablemente sólo se pueda dar en grandes instalaciones (Teradata en equipos NCR, Mainframes IBM, etc.); cabe mencionar que el mercado del hardware hoy en día apunta hacia mantener recursos escalables de hardware (procesadores, discos, memoria RAM) en grandes "boxes" (Blade, por ejemplo), a los que luego se los particiona lógicamente en servidores o particiones virtuales (LPARs, VMWare, etc.), con lo cual las capas lógicas que tiene que atravesar un usuario de base de datos hasta llegar al almacenamiento físico son las siguientes:



Por tanto, en una instalación similar a la que presentamos en la pirámide anterior, sería imposible que un DBMS tenga control real sobre el disco rígido en dónde almacena archivos.

Se puede trabajar con archivos en disco utilizando las facilidades del sistema operativo (File System) o trabajar a nivel físico con System Calls de nivel de sistema de archivos "raw". Dependiendo del producto en particular se podría llegar a elegir el tipo de manejo de archivos de disco. Por lo general, se trabajará con cualquiera de los tipos de acceso a disco guardando registros con una estructura determinada.

Lo que generalmente es una constante entre distintas arquitecturas, es que la unidad de transferencia leída o escrita de un disco, se llama memoria y generalmente es una potencia de dos y es medida en kilobytes (muchas veces 4Kb u 8Kb). En estas páginas, se pueden almacenar uno o más registros dependiendo del tamaño de registro, o quizás el tamaño de página no alcance a almacenar un registro, por lo que puede haber menos de un registro en una página determinada. Los registros dentro de una página se identifican por un id único que nada tiene que ver con la clave primaria de la tabla a la que pertenece, sino que es una abstracción que generalmente utilizan los DBMS para identificar unívocamente un registro. La identificación es generalmente unívoca a nivel base de datos (no sólo a nivel de tabla cómo

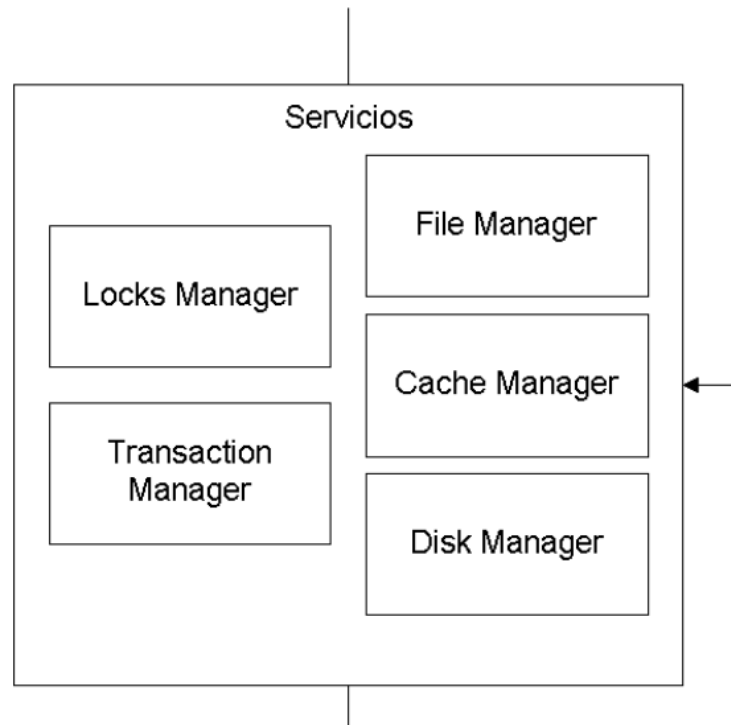
podría ser una clave primaria). Llamaremos a este identificador "Row Id". Mediante el Row Id se puede obtener la página que contiene al registro en el disco.

Un archivo de registros agrupa muchas páginas de registros. Cabe destacar que hablamos de archivo de registros específicamente, ya que un índice es también un archivo, pero no de registros. No necesariamente tiene archivos de una única tabla, y quizás tampoco contenga todos los registros de una tabla, sino que se encuentren en varios registros.

La organización de los archivos es diferente en cada DBMS, pero básicamente podemos hablar de:

- **Archivos de registro secuenciales (o heap - montículo -):** Los registros se almacenan en las páginas en el orden en que son insertados.
- **Archivos agrupados u ordenados:** Los registros se almacenan en las páginas en un orden en particular (por una relación de orden declarada sobre uno - o una lista - de atributos de la tabla en cuestión).

Recordando el esquema de arquitectura presentado previamente, si hacemos un "zoom", podemos ver que en el sector de servicios se encuentran:



El File Manager, el Cache Manager y el Disk Manager. El rol de cada uno se describe a continuación:

- El Cache Manager se encarga de gestionar espacio en memoria (RAM) para almacenar temporalmente los datos a procesar.
- El File Manager puede necesitar procesar un registro en particular, para lo cual pide el registro al Cache Manager especificando el Row Id. El Cache Manager retornará la página que contiene el registro pedido. Si el mismo no está en memoria, el Cache Manager lo traerá desde el disco. También administra todos los archivos de la base de datos, las páginas que hay en

cada uno y el espacio libre en cada página de cada archivo. Así como de la creación misma de los archivos y de su destrucción. También de la inserción, modificación y eliminación de páginas dentro de los mismos.

- El Disk Manager gestiona el espacio libre y ocupado del disco. De modo que cuándo el File Manager necesita espacio para un nuevo registro, el Disk Manager se encarga de administrarlo y también lleva una traza de las páginas que están siendo utilizadas por el File Manager, ya que, si se necesita eliminar, reutilizará el espacio cuándo sea necesario.

Las operaciones que se pueden realizar con archivos se pueden describir de la siguiente manera:

- **Abrir:** Localiza el archivo y genera un puntero al mismo, por el cual se puede acceder a las siguientes funciones.
- **Cerrar:** desliga el archivo del proceso actual, y libera los recursos utilizados para utilizarlo.
- **Siguiente:** Devuelve el siguiente registro (de tamaño y tipo especificado al llamar a la función), y mueve el puntero de archivo al siguiente.
- **Scan:** que es en realidad un bucle de operaciones "siguiente", buscando el registro en particular.
- **BOF:** Principio del archivo
- **EOF:** Fin del archivo

Bibliografía y referencias

- "Sistemas de Bases de Datos - Conceptos fundamentales" - Elmasti/Navathe - Segunda Edición Addison Wesley Iberoamericana.
- "Sistemas de Gestión de Bases de Datos" - Ramakrishnan/Gerke - Tercera edición Ed. McGraw Hill
- "Introducción a las bases de datos Relacionales" - Mendelzohn/Ale - Pearson Education